



ENGINEERING COLLEGE

Approved by AICTE New Delhi & Affiliated by VTU, Belagavi
Doddakallasandra, Off Kanakapura Main Road,
Next to Gokulam Apartment, Bangalore - 560 062.

Affiliated to
Visvesvaraya Technological University



Department of Computer Science & Engineering
(IoT and Cyber Security including Block chain Technology)

LAB MANUAL

DATABASE MANAGEMENT SYSTEM

SUBJECT CODE: BCS403

Name: _____

Branch: _____

Semester: _____

Website: www.cityengineeringcollege.ac.in

Email: info@cityengineeringcollege.ac.in

hod_icbt@cityengineeringcollege.ac.in



CITY ENGINEERING COLLEGE

VISION

“Making Remarkable Contribution by Disseminating Knowledge on Emerging Trends in Engineering and Technology through various Programmes, Innovation and Research so as to Excel in Quality both at National and International level and to provide Career Guidance & Training for Employment”.

MISSION

- M1. To encourage Knowledge Acquisition and Foster Innovation & Research.
- M2: To Prepare Students for Immediate Employment, leading to Technological and Socio-economical growth.
- M3. To Provide Guidance for a Productive Career under various programmes.



CITY ENGINEERING COLLEGE

Department of Computer Science & Engineering

(IoT and Cyber Security including Block chain Technology)

VISION

“To be a leading force in the fields of IoT, Cybersecurity, and Block chain Technology, creating a secure, trustworthy, and decentralized digital world by educating the next generation of engineers who will innovate and create resilient technological landscapes for the benefit of society.”

MISSION

- M1- Provide a comprehensive education that merges core computer science with specialized, hands-on training in IOT, cybersecurity, and blockchain.
- M2- Product technically proficient, ethical, and forward-thinking graduates.
- M3- Prepare graduates to solve complex challenges and secure the digital infrastructure of the future.
- M4- Maintain a commitment to continuous research and an industry- relevant curriculum.



CITY ENGINEERING COLLEGE

Department of Computer Science & Engineering

(IoT and Cyber Security including Block chain Technology)

Program Educational Objectives

PEO1: Graduates will excel in computer science while specializing in IoT, cyber security, and blockchain technologies to design and develop secure, innovative, and impactful technological solutions.

PEO2: Graduates will apply their technical skills to tackle real-world issues, exhibit ethical leadership, and make positive societal contributions by advancing secure, privacy-focused, and socially responsible technology.

PEO3: Graduates will engage in lifelong learning, research, and innovation to address complex challenges in digital infrastructure, fostering advancements that support industry and societal growth.

Program Specific Objectives

PSO1: Apply knowledge of computer science with specialized skills in IoT, cybersecurity, and blockchain to design, implement, and manage secure, intelligent, and decentralized systems.

PSO2: Demonstrate the ability to innovate, research, and develop industry-relevant solutions that strengthen digital infrastructure and address emerging challenges in the technological landscape.



CITY ENGINEERING COLLEGE

Department of Computer Science & Engineering (IoT and Cyber Security including Block chain Technology)

Program Outcomes (PO's)

1. **Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and engineering. specialization to the solution of complex engineering problems.
2. **Problem analysis:** Identify, formulate, research literature, and analyze engineering problems to arrive at substantiated conclusions using first principles of mathematics, natural, and engineering sciences.
3. **Design/development of solutions:** Design solutions for complex engineering problems and design system components, processes to meet the specifications with consideration for the public health and safety, and the cultural, societal, and environmental considerations.
4. **Conduct investigations of complex problems:** Use research-based knowledge including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
5. **Modern tool usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
6. **The engineer and society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal, and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

7. **Environment and sustainability:** Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

8. **Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

9. **Individual and team work:** Function effectively as an individual, and as a member or leader in teams, and in multidisciplinary settings.

10. **Communication:** Communicate effectively with the engineering community and with society at large. Be able to comprehend and write effective reports documentation. Make effective presentations, and give and receive clear instructions.

11. **Project management and finance:** Demonstrate knowledge and understanding of engineering and management principles and apply these to one's own work, as a member and leader in a team. Manage projects in multidisciplinary environments.

12. **Life-long learning:** Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.



CITY ENGINEERING COLLEGE

Department of Computer Science & Engineering

(IoT and Cyber Security including Block chain Technology)

GENERAL LABORATORY INSTRUCTIONS

1. Students are advised to come to the laboratory at least 5 minutes before (to starting time), those who come after 5 minutes will not be allowed into the lab.
2. Plan your task properly much before to the commencement, come prepared to the lab with the synopsis/ program / experiment details.
3. Student should enter into the laboratory with: a. Laboratory observation notes with all the details (Problem statement, Aim, Algorithm, Procedure, Program, Expected Output, etc.,) filled in for the lab session. b. Laboratory Record updated up to the last session experiments and other utensils (if any) needed in the lab. c. Proper Dress code and Identity card.
4. Sign in the laboratory login register, write the TIME-IN, and occupy the computer system allotted to you by the faculty.
5. Execute your task in the laboratory, and record the results / output in the lab observation note book, and get certified by the concerned faculty.
6. All the students should be polite and cooperative with the laboratory staff, must maintain the discipline and decency in the laboratory.
7. Computer labs are established with sophisticated and high end branded systems, which should be utilized properly.
8. Students / Faculty must keep their mobile phones in SWITCHED OFF mode during the lab sessions. Misuse of the equipment, misbehaviors with the staff and systems etc., will attract severe punishment.
9. Students must take the permission of the faculty in case of any urgency to go out if any body found loitering outside the lab / class without permission during working hours will be treated seriously and punished appropriately.
10. Students should LOG OFF/ SHUT DOWN the computer system before he/she leaves the lab after completing the task (experiment) in all aspects. He/she must ensure the system / seat is kept properly.



CITY ENGINEERING COLLEGE

Department of Computer Science & Engineering

(IoT and Cyber Security including Block chain Technology)

CODE OF CONDUCT FOR THE LABORATORY

- All students must observe the dress code while in the laboratory
- Footwear is NOT allowed
- Foods, drinks and smoking are NOT allowed
- All bags must be left at the indicated place
- The lab timetable must be strictly followed
- Be PUNCTUAL for your laboratory session
- All programs must be completed within the given time
- Noise must be kept to a minimum
- Workspace must be kept clean and tidy at all time
- All students are liable for any damage to system due to their own negligence
- Students are strictly PROHIBITED from taking out any items from the laboratory
- Report immediately to the lab programmer if any damages to equipment

BEFORE LEAVING LAB:

- Arrange all the equipment and chairs properly.
- Turn off / shut down the systems before leaving.
- Please check the laboratory notice board regularly for updates.

DATABASE MANAGEMENT SYSTEM		Semester	4
Course Code	BCS403	CIE Marks	50
Teaching Hours/Week (L:T:P: S)	3:0:2:0	SEE Marks	50
Total Hours of Pedagogy	40 hours Theory + 8-10 Lab slots	Total Marks	100
Credits	04	Exam Hours	
Examination nature (SEE)	Theory		

Course objectives:

- To Provide a strong foundation in database concepts, technology, and practice.
- To Practice SQL programming through a variety of database problems.
- To Understand the relational database design principles.
- To Demonstrate the use of concurrency and transactions in database.
- To Design and build database applications for real world problems.
- To become familiar with database storage structures and access techniques.

Teaching-Learning Process

These are sample Strategies, which teachers can use to accelerate the attainment of the various course outcomes.

1. Lecturer method (L) needs not to be only a traditional lecture method, but alternative effective teaching methods could be adopted to attain the outcomes.
2. Use of Video/Animation to explain functioning of various concepts.
3. Encourage collaborative (Group Learning) Learning in the class.
4. Ask at least three HOT (Higher order Thinking) questions in the class, which promotes critical thinking.
5. Adopt Problem Based Learning (PBL), which fosters students' Analytical skills, develop design thinking skills such as the ability to design, evaluate, generalize, and analyze information rather than simply recall it.
6. Introduce Topics in manifold representations.
7. Show the different ways to solve the same problem with different circuits/logic and encourage the students to come up with their own creative ways to solve them.
8. Discuss how every concept can be applied to the real world - and when that's possible, it helps improve the students' understanding
9. Use any of these methods: Chalk and board, Active Learning, Case Studies

PRACTICAL COMPONENT OF IPCC *(May cover all / major modules)*

Sl.NO	Experiments
1	Create a table called Employee & execute the following. Employee(EMPNO,ENAME,JOB, MANAGER_NO, SAL, COMMISSION) 1. Create a user and grant all permissions to the user. 2. Insert the any three records in the employee table contains attributes EMPNO,ENAME JOB, MANAGER_NO, SAL, COMMISSION and use rollback. Check the result. 3. Add primary key constraint and not null constraint to the employee table. 4. Insert null values to the employee table and verify the result.
2	Create a table called Employee that contain attributes EMPNO,ENAME,JOB, MGR,SAL & execute the following. 1. Add a column commission with domain to the Employee table. 2. Insert any five records into the table. 3. Update the column details of job 4. Rename the column of Employee table using alter command. 5. Delete the employee whose Empno is 105.
3	Queries using aggregate functions(COUNT,AVG,MIN,MAX,SUM),Group by,Order by. Employee(E_id, E_name, Age, Salary) 1. Create Employee table containing all Records E_id, E_name, Age, Salary. 2. Count number of employee names from employee table 3. Find the Maximum age from employee table. 4. Find the Minimum age from employee table. 5. Find salaries of employee in Ascending Order. 6. Find grouped salaries of employees.
4	Create a row level trigger for the customers table that would fire for INSERT or UPDATE or DELETE operations performed on the CUSTOMERS table. This trigger will display the salary difference between the old & new Salary. CUSTOMERS(ID,NAME,AGE,ADDRESS,SALARY)
5	Create cursor for Employee table & extract the values from the table. Declare the variables ,Open the cursor & extract the values from the cursor. Close the cursor. Employee(E_id, E_name, Age, Salary)
6	Write a PL/SQL block of code using parameterized Cursor, that will merge the data available in the newly created table N_RollCall with the data available in the table O_RollCall. If the data in the first table already exist in the second table then that data should be skipped.
7	Install an Open Source NoSQL Data base MangoDB & perform basic CRUD(Create, Read, Update & Delete) operations. Execute MangoDB basic Queries using CRUD operations.
Course outcomes (Course Skill Set): At the end of the course, the student will be able to: • Describe the basic elements of a relational database management system • Design entity relationship for the given scenario. • Apply various Structured Query Language (SQL) statements for database manipulation. • Analyse various normalization forms for the given application. • Develop database applications for the given real world problem. • Understand the concepts related to NoSQL databases.	
Assessment Details (both CIE and SEE)	

minimum passing mark for the CIE is 40% of the maximum marks (20 marks out of 50) and for the SEE minimum passing mark is 35% of the maximum marks (18 out of 50 marks). A student shall be deemed to have satisfied the academic requirements and earned the credits allotted to each subject/ course if the student secures a minimum of 40% (40 marks out of 100) in the sum total of the CIE (Continuous Internal Evaluation) and SEE (Semester End Examination) taken together.

CIE for the theory component of the IPCC (maximum marks 50)

- IPCC means practical portion integrated with the theory of the course.
- CIE marks for the theory component are **25 marks** and that for the practical component is **25 marks**.
- 25 marks for the theory component are split into **15 marks** for two Internal Assessment Tests (Two Tests, each of 15 Marks with 01-hour duration, are to be conducted) and **10 marks** for other assessment methods mentioned in 22OB4.2. The first test at the end of 40-50% coverage of the syllabus and the second test after covering 85-90% of the syllabus.
- Scaled-down marks of the sum of two tests and other assessment methods will be CIE marks for the theory component of IPCC (that is for **25 marks**).
- The student has to secure 40% of 25 marks to qualify in the CIE of the theory component of IPCC.

CIE for the practical component of the IPCC

- **15 marks** for the conduction of the experiment and preparation of laboratory record, and **10 marks** for the test to be conducted after the completion of all the laboratory sessions.
- On completion of every experiment/program in the laboratory, the students shall be evaluated including viva-voce and marks shall be awarded on the same day.
- The CIE marks awarded in the case of the Practical component shall be based on the continuous evaluation of the laboratory report. Each experiment report can be evaluated for 10 marks. Marks of all experiments' write-ups are added and scaled down to **15 marks**.
- The laboratory test (**duration 02/03 hours**) after completion of all the experiments shall be conducted for 50 marks and scaled down to **10 marks**.
- Scaled-down marks of write-up evaluations and tests added will be CIE marks for the laboratory component of IPCC for **25 marks**.
- The student has to secure 40% of 25 marks to qualify in the CIE of the practical component of the IPCC.

SEE for IPCC

Theory SEE will be conducted by University as per the scheduled timetable, with common question papers for the course (**duration 03 hours**)

1. The question paper will have ten questions. Each question is set for 20 marks.
2. There will be 2 questions from each module. Each of the two questions under a module (with a maximum of 3 sub-questions), **should have a mix of topics** under that module.
3. The students have to answer 5 full questions, selecting one full question from each module.
4. Marks scored by the student shall be proportionally scaled down to 50 Marks

The theory portion of the IPCC shall be for both CIE and SEE, whereas the practical portion will have a CIE component only. Questions mentioned in the SEE paper may include questions from the practical component.

Suggested Learning Resources:

Text Books:

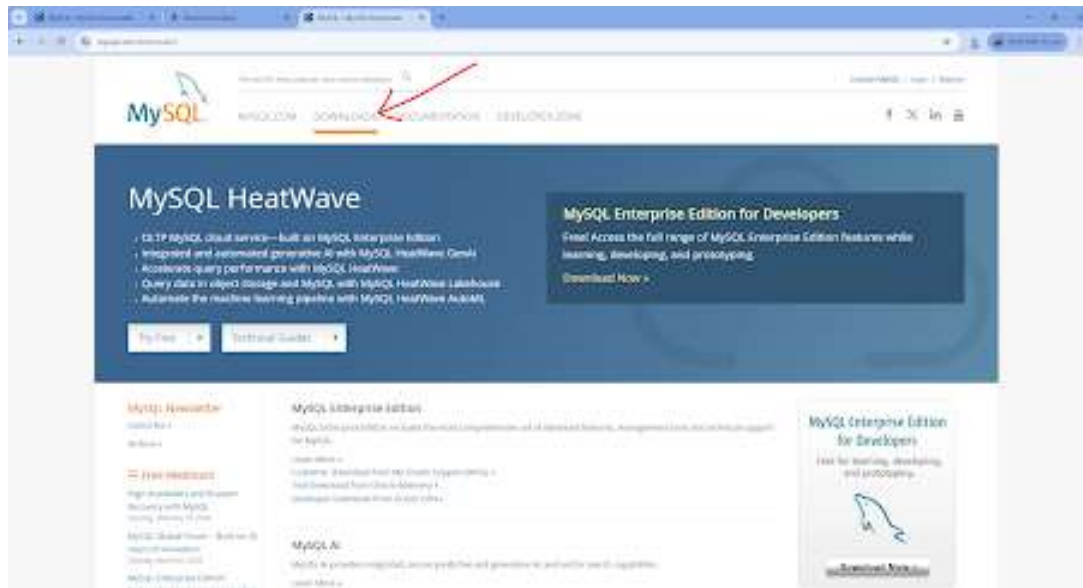
1. Fundamentals of Database Systems, Ramez Elmasri and Shamkant B. Navathe, 7th Edition, 2017, Pearson.
2. Database management systems, Ramakrishnan, and Gehrke, 3rd Edition, 2014, McGraw Hill

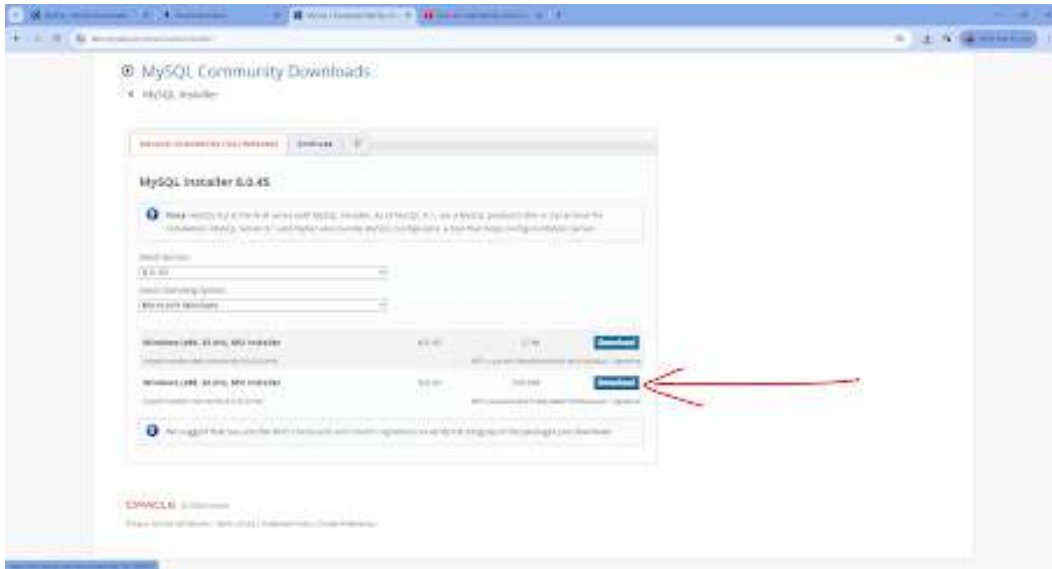
Activity Based Learning (Suggested Activities in Class)/ Practical Based learning

Mini Project:

- Project Based Learning

SQL INSTALLATION _ WINDOWS OS





install MySQL on Windows:

1. Download MySQL

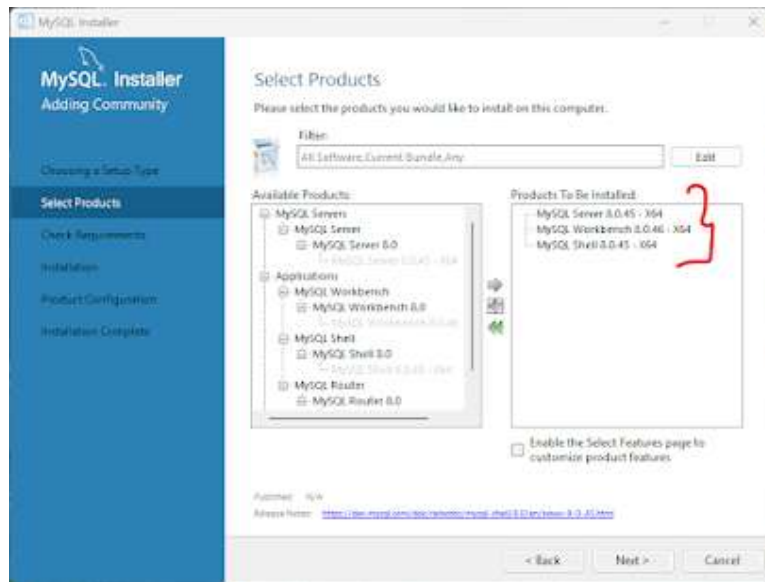
- Go to: <https://dev.mysql.com/downloads/>
- Click **MySQL Installer for Windows**
- Download the **Windows (x86, 32-bit), MSI Installer**
- Click **No thanks, just start my download**

2. Run the Installer

- Open the downloaded .msi file
- Click **Yes** if prompted

3. Choose Setup Type

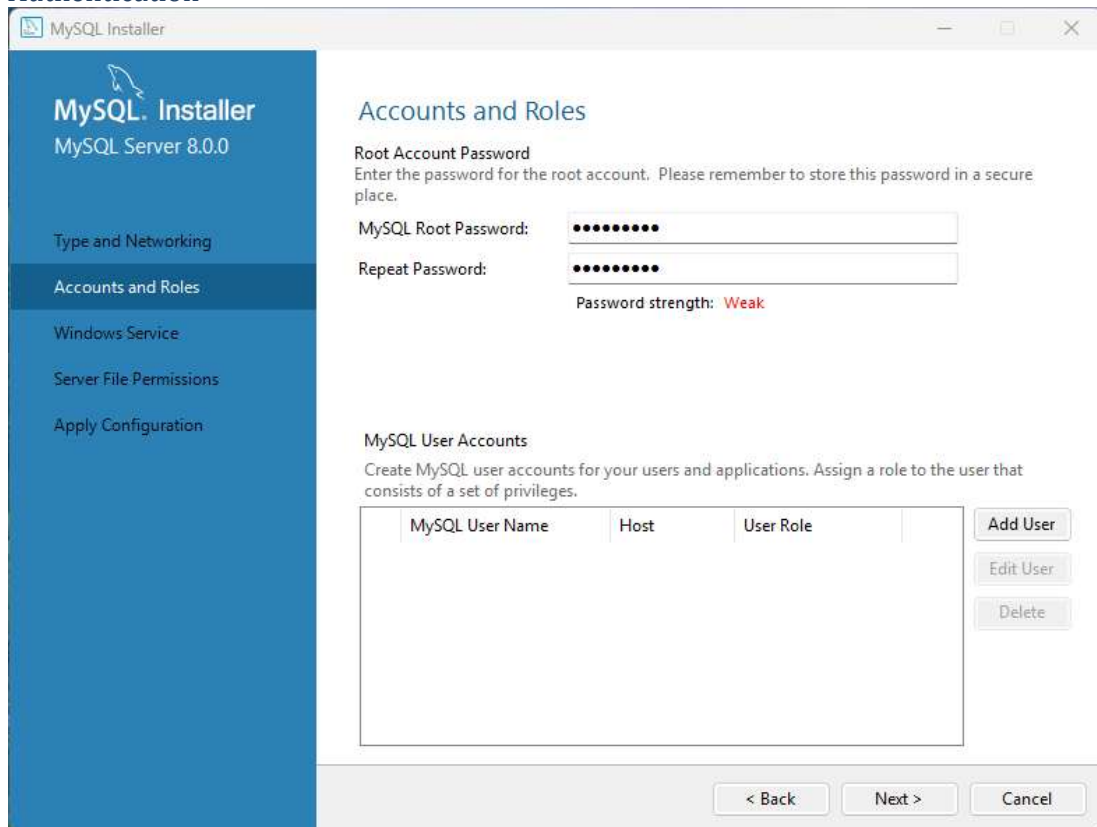
- Select **Developer Default** (recommended)
- Click **Next**
- Click **Execute** to install required files



4. Configure MySQL

- Choose **Standalone MySQL Server**
- Click **Next**

5. Authentication



- Select **Use Strong Password Encryption**
- Set a **root password**
- Click **Next**

6. Windows Service

- Keep default settings
- Click **Next**

7. Apply Configuration

- Click **Execute**
- Wait until completed
- Click **Finish**

8. Open MySQL

- Open **MySQL Workbench**
- Enter root password
- Start using MySQL 

EX NO 1

Create a table called Employee & execute the following.

Employee(EMPNO,ENAME, JOB, MANAGER_NO, SAL, COMMISSION)

1. Create a user and grant all permissions to the user.
2. Insert the any three records in the employee table contains attributes EMPNO,ENAME JOB, MANAGER_NO, SAL, COMMISSION and use rollback.
Check the result.
3. Add primary key constraint and not null constraint to the employee table.
Insert null values to the employee table and verify the result.

I. Create a user and grant all permissions to the user.

CREATE USER 'emp_jones'@'localhost' IDENTIFIED BY 'emp123';

GRANT ALL PRIVILEGES ON *.* TO 'emp_user'@'localhost' WITH GRANT OPTION;

FLUSH PRIVILEGES;

What this does:

1 Create a user

```
CREATE USER 'emp_user'@'localhost' IDENTIFIED BY 'emp123';
```

EXPLANATION

emp_user → username

@'localhost' → user can log in only from the local machine

IDENTIFIED BY 'emp123' → sets the password

2 Grant all privileges

```
GRANT ALL PRIVILEGES ON *.* TO 'emp_user'@'localhost' WITH GRANT OPTION;
```

EXPLANATION

- **ALL PRIVILEGES** → gives all permissions (SELECT, INSERT, UPDATE, DELETE, CREATE, DROP, etc.)
- ***.*** → applies to **all databases and all tables**

- **WITH GRANT OPTION** → allows emp_user to grant permissions to other users

Result:

emp_user has **full control** over the entire MySQL server.

⚠ *Note:* This is fine for **lab/practice**, but unsafe for production.

3 Apply changes immediately

```
FLUSH PRIVILEGES;
```

EXPLANATION

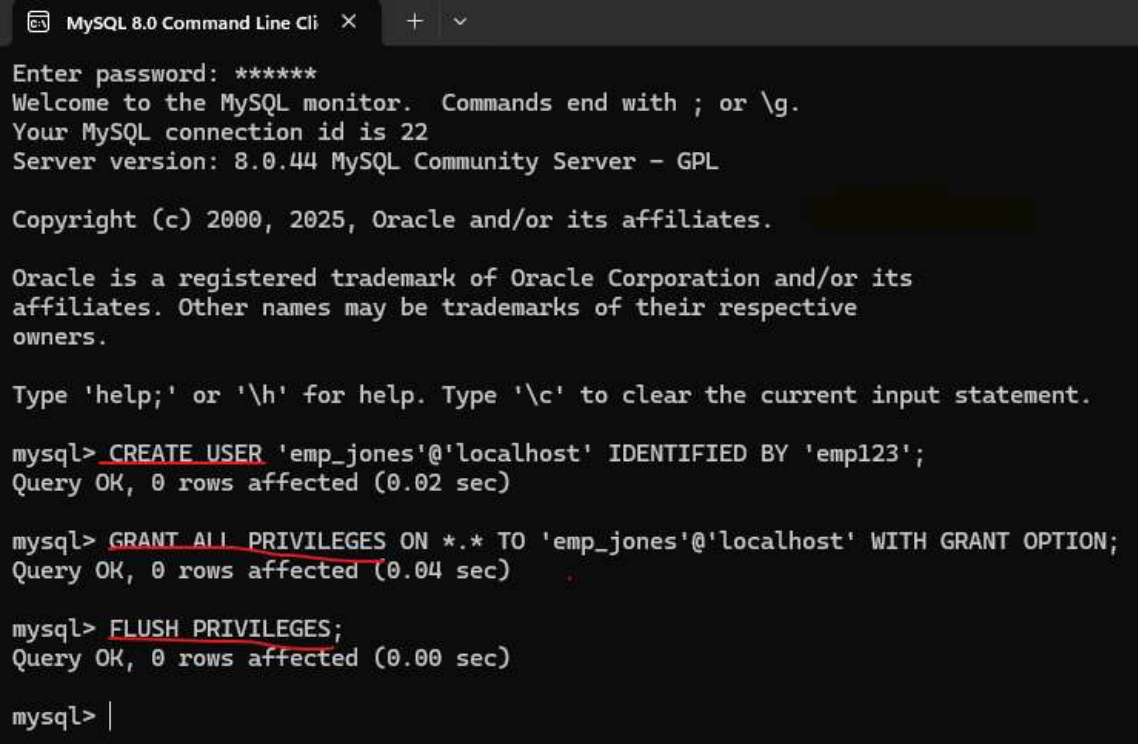
- Reloads the privilege tables
- Ensures the new permissions take effect immediately

Result:

No need to restart MySQL — changes are active right away.

Result:

A new MySQL user named emp_user is created with password emp123.



```
MySQL 8.0 Command Line Cli  X  +  v
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 22
Server version: 8.0.44 MySQL Community Server - GPL

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE USER 'emp_jones'@'localhost' IDENTIFIED BY 'emp123';
Query OK, 0 rows affected (0.02 sec)

mysql> GRANT ALL PRIVILEGES ON *.* TO 'emp_jones'@'localhost' WITH GRANT OPTION;
Query OK, 0 rows affected (0.04 sec)

mysql> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.00 sec)

mysql> |
```

II. Insert the any three records in the employee table contains attributes EMPNO,ENAME JOB, MANAGER NO, SAL, COMMISSION and use rollback. Check the result.

step 1:

```
CREATE DATABASE joda_db;
```

```
mysql> CREATE DATABASE joda_db;  
Query OK, 1 row affected (0.10 sec)  
  
mysql> |
```

step2:

```
USE joda_db;
```

```
mysql> USE joda_db;  
Database changed  
mysql> |
```

step3 :

Insert the any three records in the employee table contains attributes EMPNO,ENAME JOB, MANAGER_NO, SAL, COMMISSION

```
CREATE TABLE EMPLOYEE (  
    EMPNO          INT PRIMARY KEY,  
    ENAME          VARCHAR(50) ,  
    JOB            VARCHAR(50) ,  
    MANAGER_NO     INT ,  
    SAL            DECIMAL(10,2) ,  
    COMMISSION     DECIMAL(10,2)  
);
```

```
mysql> CREATE TABLE Employee (  
->    EMPNO          INT,  
->    ENAME          VARCHAR(20),  
->    JOB            VARCHAR(20),  
->    MANAGER_NO     INT,  
->    SAL            DECIMAL(10,2),  
->    COMMISSION     DECIMAL(10,2)  
-> );  
Query OK, 0 rows affected (0.02 sec)  
  
mysql> |
```

use rollback (Enter Values). Check the result

```
INSERT INTO Employee VALUES (101, 'John', 'Manager', NULL,
50000, 5000);
INSERT INTO Employee VALUES (102, 'vikki', 'Clerk', 101, 25000,
2000);
INSERT INTO Employee VALUES (103, 'Raj', 'Salesman', 101,
30000, 3000);
```

```
mysql>
mysql> INSERT INTO Employee VALUES (101, 'John', 'Manager', NULL, 50000, 5000);
Query OK, 1 row affected (0.03 sec)

mysql> INSERT INTO Employee VALUES (102, 'vikki', 'Clerk', 101, 25000, 2000);
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO Employee VALUES (103, 'Raj', 'Salesman', 101, 30000, 3000);
Query OK, 1 row affected (0.00 sec)

mysql>
mysql>
```

show table... etc

```
SELECT * FROM Employee;
```

```
mysql>
mysql> SHOW TABLES;
+-----+
| Tables_in_joda_db |
+-----+
| employee          |
+-----+
1 row in set (0.01 sec)

mysql> show db
->
->
-> DESC Employee;
ERROR 1064 (42000): You have an error in your SQL syntax; check the

DESC Employee' at line 1
mysql> DESC Employee;
+-----+-----+-----+-----+-----+-----+
| Field      | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| EMPNO      | int           | YES  |     | NULL    |       |
| ENAME      | varchar(20)   | YES  |     | NULL    |       |
| JOB        | varchar(20)   | YES  |     | NULL    |       |
| MANAGER_NO | int           | YES  |     | NULL    |       |
| SAL        | decimal(10,2) | YES  |     | NULL    |       |
| COMMISSION | decimal(10,2) | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
6 rows in set (0.03 sec)

mysql> SELECT * FROM Employee;
+-----+-----+-----+-----+-----+-----+
| EMPNO | ENAME | JOB      | MANAGER_NO | SAL      | COMMISSION |
+-----+-----+-----+-----+-----+-----+
| 101   | John  | Manager  | NULL       | 50000.00 | 5000.00    |
| 102   | vikki | Clerk    | 101        | 25000.00 | 2000.00    |
| 103   | Raj   | Salesman | 101        | 30000.00 | 3000.00    |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> |
```

III, 3. Add primary key constraint and not null constraint to the employee table.Step 1: Make EMPNO the Primary Key

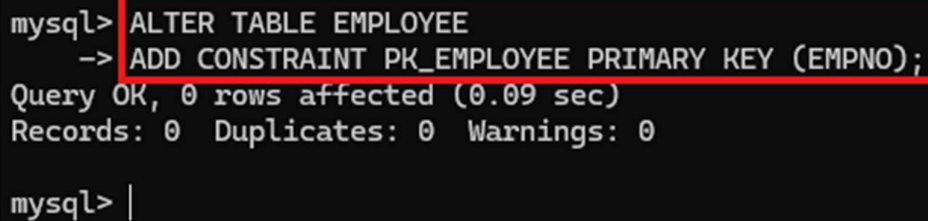
This means **no duplicate EMPNO** and **no empty EMPNO**.

```
ALTER TABLE EMPLOYEE  
ADD CONSTRAINT PK_EMPLOYEE PRIMARY KEY (EMPNO);
```

note:

This line tells the database:
“EMPNO is the main ID for each employee.”

```
ALTER TABLE EMPLOYEE  
  
ADD PRIMARY KEY (EMPNO);
```

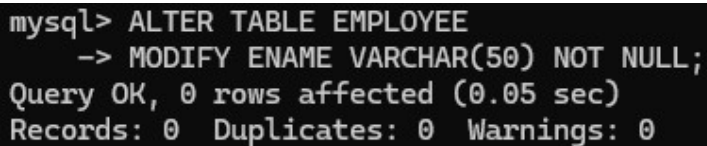


```
mysql> ALTER TABLE EMPLOYEE  
-> ADD CONSTRAINT PK_EMPLOYEE PRIMARY KEY (EMPNO);  
Query OK, 0 rows affected (0.09 sec)  
Records: 0 Duplicates: 0 Warnings: 0  
  
mysql> |
```

Step 2: Make columns NOT NULL

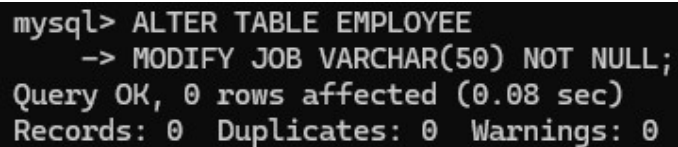
This means these columns **must have values**.

```
ALTER TABLE EMPLOYEE  
MODIFY ENAME VARCHAR(50) NOT NULL;
```



```
mysql> ALTER TABLE EMPLOYEE  
-> MODIFY ENAME VARCHAR(50) NOT NULL;  
Query OK, 0 rows affected (0.05 sec)  
Records: 0 Duplicates: 0 Warnings: 0
```

```
ALTER TABLE EMPLOYEE  
MODIFY JOB VARCHAR(50) NOT NULL;
```



```
mysql> ALTER TABLE EMPLOYEE  
-> MODIFY JOB VARCHAR(50) NOT NULL;  
Query OK, 0 rows affected (0.08 sec)  
Records: 0 Duplicates: 0 Warnings: 0
```

```
ALTER TABLE EMPLOYEE  
MODIFY SAL DECIMAL(10,2) NOT NULL;
```

```
mysql> ALTER TABLE EMPLOYEE
      -> MODIFY SAL DECIMAL(10,2) NOT NULL;
Query OK, 0 rows affected (0.05 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

🔗 These lines tell the database:

“ENAME, JOB, and SAL cannot be empty/Null.”

Before Make columns NOT NULL

Field	Type	Null	Key	Default	Extra
EMPNO	int	YES		NULL	
ENAME	varchar(50)	YES		NULL	
JOB	varchar(50)	YES		NULL	
MANAGER_NO	int	YES		NULL	
SAL	decimal	YES		NULL	
COMMISSION	decimal	YES		NULL	

After Make columns NOT NULL

Field	Type	Null	Key	Default	Extra
EMPNO	int	NO	PRI	NULL	
ENAME	varchar(50)	NO		NULL	
JOB	varchar(50)	NO		NULL	
MANAGER_NO	int	YES		NULL	
SAL	decimal	NO		NULL	
COMMISSION	decimal	YES		NULL	

IV, 4. Insert null values to the employee table and verify the result.

Note:

can't use this command for Empno, Ename, Job, Sal because we changed these as NOT NULL.

So we do remaining Managerno and Commission.

 Try inserting NULL into NOT NULL columns

sql

NOT POSSIBLE

```
INSERT INTO EMPLOYEE (EMPNO, ENAME, JOB, SAL)
VALUES (201, NULL, 'Clerk', 15000);
```

Result:  Error

sql

```
ERROR 1048 (23000): Column 'ENAME' cannot be null
```

SOLUTION:

Insert NULL into nullable columns (works)

```
INSERT INTO EMPLOYEE (EMPNO, ENAME, JOB, SAL, MANAGER_NO,
COMMISSION)
```

```
VALUES (202, 'Ravi', 'Sales', 18000, NULL, NULL);
```

```
mysql> INSERT INTO EMPLOYEE (EMPNO, ENAME, JOB, SAL, MANAGER_NO, COMMISSION)
-> VALUES (202, 'Ravi', 'Sales', 18000, NULL, NULL);
Query OK, 1 row affected (0.01 sec)
mysql> |
```

Verify the result:

```
SELECT * FROM EMPLOYEE WHERE EMPNO = 202;
```

```
mysql> SELECT * FROM EMPLOYEE WHERE EMPNO = 202;
+-----+-----+-----+-----+-----+-----+
| EMPNO | ENAME | JOB   | MANAGER_NO | SAL      | COMMISSION |
+-----+-----+-----+-----+-----+-----+
| 202   | Ravi  | Sales | NULL       | 18000.00 | NULL       |
+-----+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)
mysql> |
```

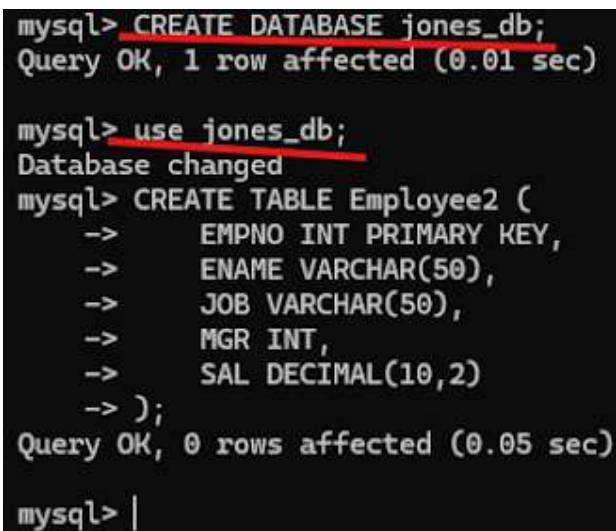
EX 2:

Create a table called Employee that contain attributes EMPNO,ENAME,JOB, MGR,SAL & execute the following.

1. Add a column commission with domain to the Employee table.
2. Insert any five records into the table.
3. Update the column details of job
4. Rename the column of Employee table using alter command.
5. Delete the employee whose Empno is 105.

Create Table Employee:

```
CREATE TABLE Employee2 (  
    EMPNO INT PRIMARY KEY,  
    ENAME VARCHAR(50),  
    JOB VARCHAR(50),  
    MGR INT,  
    SAL DECIMAL(10,2)  
);
```



```
mysql> CREATE DATABASE jones_db;  
Query OK, 1 row affected (0.01 sec)  
  
mysql> use jones_db;  
Database changed  
mysql> CREATE TABLE Employee2 (  
-> EMPNO INT PRIMARY KEY,  
-> ENAME VARCHAR(50),  
-> JOB VARCHAR(50),  
-> MGR INT,  
-> SAL DECIMAL(10,2)  
-> );  
Query OK, 0 rows affected (0.05 sec)  
  
mysql> |
```


Add Column COMMISSION with Domain

(Example: Commission must be numeric and greater than or equal to 0)

```
mysql> ALTER TABLE Employee2
-> ADD COMMISSION DECIMAL(10,2) CHECK (COMMISSION >= 0);
Query OK, 0 rows affected (0.09 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> select * from employee2;
Empty set (0.00 sec)

mysql> DESC employee2
->
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| EMPNO | int | NO | PRI | NULL | |
| ENAME | varchar(50) | YES | | NULL | |
| JOB | varchar(50) | YES | | NULL | |
| MGR | int | YES | | NULL | |
| SAL | decimal(10,2) | YES | | NULL | |
| COMMISSION | decimal(10,2) | YES | | NULL | |
+-----+-----+-----+-----+-----+
6 rows in set (0.01 sec)

mysql> |
```

Insert Five Records

```
INSERT INTO Employee2 VALUES (101, 'John', 'Manager', 100, 50000, 5000);
INSERT INTO Employee2 VALUES (102, 'Smith', 'Clerk', 101, 25000, 2000);
INSERT INTO Employee2 VALUES (103, 'David', 'Analyst', 101, 40000, 3000);
INSERT INTO Employee2 VALUES (104, 'Sara', 'Salesman', 102, 30000, 2500);
INSERT INTO Employee2 VALUES (105, 'Linda', 'Clerk', 101, 22000, 1500);
```

```
mysql> INSERT INTO Employee2 VALUES (101, 'John', 'Manager', 100, 50000, 5000);
Query OK, 1 row affected (0.03 sec)

mysql> INSERT INTO Employee2 VALUES (102, 'Smith', 'Clerk', 101, 25000, 2000);
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO Employee2 VALUES (103, 'David', 'Analyst', 101, 40000, 3000);
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO Employee2 VALUES (104, 'Sara', 'Salesman', 102, 30000, 2500);
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO Employee2 VALUES (105, 'Linda', 'Clerk', 101, 22000, 1500);
Query OK, 1 row affected (0.01 sec)

mysql> select * from employee2;
+-----+-----+-----+-----+-----+-----+
| EMPNO | ENAME | JOB | MGR | SAL | COMMISSION |
+-----+-----+-----+-----+-----+-----+
| 101 | John | Manager | 100 | 50000.00 | 5000.00 |
| 102 | Smith | Clerk | 101 | 25000.00 | 2000.00 |
| 103 | David | Analyst | 101 | 40000.00 | 3000.00 |
| 104 | Sara | Salesman | 102 | 30000.00 | 2500.00 |
| 105 | Linda | Clerk | 101 | 22000.00 | 1500.00 |
+-----+-----+-----+-----+-----+
5 rows in set (0.00 sec)

mysql> |
```

Update the Column Details of JOB

(Example: Change JOB of employee 102)

```

UPDATE Employee2
SET JOB = 'Senior Clerk'
WHERE EMPNO = 102;

```

```

mysql> select * from employee2;
+-----+-----+-----+-----+-----+-----+
| EMPNO | ENAME | JOB      | MGR | SAL      | COMMISSION |
+-----+-----+-----+-----+-----+-----+
| 101   | John  | Manager  | 100 | 50000.00 | 5000.00    |
| 102   | Smith | Clerk     | 101 | 25000.00 | 2000.00    |
| 103   | David | Analyst   | 101 | 40000.00 | 3000.00    |
| 104   | Sara  | Salesman  | 102 | 30000.00 | 2500.00    |
| 105   | Linda | Clerk     | 101 | 22000.00 | 1500.00    |
+-----+-----+-----+-----+-----+-----+
5 rows in set (0.00 sec)

mysql> UPDATE Employee2
-> SET JOB = 'Senior Clerk'
-> WHERE EMPNO = 102;
Query OK, 1 row affected (0.03 sec)
Rows matched: 1 Changed: 1 Warnings: 0

mysql> select * from employee2;
+-----+-----+-----+-----+-----+-----+
| EMPNO | ENAME | JOB      | MGR | SAL      | COMMISSION |
+-----+-----+-----+-----+-----+-----+
| 101   | John  | Manager  | 100 | 50000.00 | 5000.00    |
| 102   | Smith | Senior Clerk | 101 | 25000.00 | 2000.00    |
| 103   | David | Analyst   | 101 | 40000.00 | 3000.00    |
| 104   | Sara  | Salesman  | 102 | 30000.00 | 2500.00    |
| 105   | Linda | Clerk     | 101 | 22000.00 | 1500.00    |
+-----+-----+-----+-----+-----+-----+
5 rows in set (0.00 sec)

mysql>

```

Rename Column of Employee Table Using ALTER

(Example: Rename ENAME to EMP_NAME)

```

ALTER TABLE Employee2
RENAME COLUMN ENAME TO EMP_NAME;

```

```
mysql> select * from employee2;
```

EMPNO	ENAME	JOB	MGR	SAL	COMMISSION
101	John	Manager	100	50000.00	5000.00
102	Smith	Senior Clerk	101	25000.00	2000.00
103	David	Analyst	101	40000.00	3000.00
104	Sara	Salesman	102	30000.00	2500.00
105	Linda	Clerk	101	22000.00	1500.00

5 rows in set (0.00 sec)

```
mysql> ALTER TABLE Employee2
-> RENAME COLUMN ENAME TO EMP_NAME;
Query OK, 0 rows affected (0.02 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

updated

```
mysql> select * from employee2;
```

EMPNO	EMP_NAME	JOB	MGR	SAL	COMMISSION
101	John	Manager	100	50000.00	5000.00
102	Smith	Senior Clerk	101	25000.00	2000.00
103	David	Analyst	101	40000.00	3000.00
104	Sara	Salesman	102	30000.00	2500.00
105	Linda	Clerk	101	22000.00	1500.00

5 rows in set (0.00 sec)

```
mysql> |
```

Delete Employee Whose EMPNO is 105

DELETE FROM Employee2

WHERE EMPNO = 105;

```
mysql> select * from employee2;
```

EMPNO	EMP_NAME	JOB	MGR	SAL	COMMISSION
101	John	Manager	100	50000.00	5000.00
102	Smith	Senior Clerk	101	25000.00	2000.00
103	David	Analyst	101	40000.00	3000.00
104	Sara	Salesman	102	30000.00	2500.00
105	Linda	Clerk	101	22000.00	1500.00

5 rows in set (0.00 sec)

```
mysql> DELETE FROM Employee2
-> WHERE EMPNO = 105;
Query OK, 1 row affected (0.01 sec)
```

Delete

```
mysql> select * from employee2;
```

EMPNO	EMP_NAME	JOB	MGR	SAL	COMMISSION
101	John	Manager	100	50000.00	5000.00
102	Smith	Senior Clerk	101	25000.00	2000.00
103	David	Analyst	101	40000.00	3000.00
104	Sara	Salesman	102	30000.00	2500.00

4 rows in set (0.00 sec)

```
mysql>
```

EX no 3:

Queries using aggregate functions(COUNT,AVG,MIN,MAX,SUM),Group by,Orderby. Employee(E_id, E_name, Age, Salary) 1. Create Employee table containing all Records E_id, E_name, Age, Salary. 2. Count number of employee names from employeetable 3. Find the Maximum age from employee table. 4. Find the Minimum age from employeetable. 5. Find salaries of employee in Ascending Order. 6. Find grouped salaries of employees.

Create Employee Table

```
CREATE TABLE Employee (
    E_id INT PRIMARY KEY,
    E_name VARCHAR(50),
    Age INT,
    Salary DECIMAL(10,2)
);
```

```
mysql> CREATE DATABASE daniel_db;
Query OK, 1 row affected (0.01 sec)

mysql> use daniel_db;
Database changed
mysql> CREATE TABLE Employee (
    -> E_id INT PRIMARY KEY,
    -> E_name VARCHAR(50),
    -> Age INT,
    -> Salary DECIMAL(10,2)
    -> );
Query OK, 0 rows affected (0.02 sec)

mysql> select * from employee;
Empty set (0.00 sec)

mysql> DESC employee;
```

Field	Type	Null	Key	Default	Extra
E_id	int	NO	PRI	NULL	
E_name	varchar(50)	YES		NULL	
Age	int	YES		NULL	
Salary	decimal(10,2)	YES		NULL	

```
4 rows in set (0.00 sec)

mysql> |
```

(Insert some sample records)

```
mysql> INSERT INTO Employee VALUES (1, 'John', 25, 30000);
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO Employee VALUES (2, 'Smith', 30, 40000);
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO Employee VALUES (3, 'David', 28, 35000);
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO Employee VALUES (4, 'Sara', 35, 45000);
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO Employee VALUES (5, 'Linda', 30, 40000);
Query OK, 1 row affected (0.00 sec)

mysql> select * from employee;
+-----+-----+-----+-----+
| E_id | E_name | Age | Salary |
+-----+-----+-----+-----+
| 1 | John | 25 | 30000.00 |
| 2 | Smith | 30 | 40000.00 |
| 3 | David | 28 | 35000.00 |
| 4 | Sara | 35 | 45000.00 |
| 5 | Linda | 30 | 40000.00 |
+-----+-----+-----+-----+
5 rows in set (0.00 sec)

mysql> |
```

Count Number of Employee Names

```
SELECT COUNT(E_name) AS Total_Employees
FROM Employee;
```

```
mysql> SELECT COUNT(E_name) AS Total_Employees
-> FROM Employee;
+-----+
| Total_Employees |
+-----+
| 5 |
+-----+
1 row in set (0.00 sec)

mysql> |
```

Find Maximum Age

```
SELECT MAX(Age) AS Maximum_Age
FROM Employee;
```



```
mysql> SELECT MAX(Age) AS Maximum_Age
-> FROM Employee;
+-----+
| Maximum_Age |
+-----+
|          35 |
+-----+
1 row in set (0.03 sec)

mysql> |
```

Find Minimum Age

```
SELECT MIN(Age) AS Minimum_Age
FROM Employee;
```

```
mysql> SELECT MIN(Age) AS Minimum_Age
-> FROM Employee;
+-----+
| Minimum_Age |
+-----+
|          25 |
+-----+
1 row in set (0.00 sec)

mysql> |
```

Find Salaries in Ascending Order

```
SELECT Salary
FROM Employee
ORDER BY Salary ASC;
```

```
mysql> SELECT Salary
-> FROM Employee
-> ORDER BY Salary ASC;
+-----+
| Salary |
+-----+
| 30000.00 |
| 35000.00 |
| 40000.00 |
| 40000.00 |
| 45000.00 |
+-----+
5 rows in set (0.00 sec)

mysql> |
```

Find Grouped Salaries of Employees

(Group employees having same salary and count them)

```
SELECT Salary, COUNT(*) AS Number_of_Employees  
FROM Employee  
GROUP BY Salary;
```

```
mysql> SELECT Salary, COUNT(*) AS Number_of_Employees  
-> FROM Employee  
-> GROUP BY Salary;  
+-----+-----+  
| Salary | Number_of_Employees |  
+-----+-----+  
| 30000.00 | 1 |  
| 40000.00 | 2 |  
| 35000.00 | 1 |  
| 45000.00 | 1 |  
+-----+-----+  
4 rows in set (0.00 sec)  
  
mysql> |
```

EX :4

Create a row level trigger for the customers table that would fire for INSERT or UPDATE or DELETE operations performed on the CUSTOMERS table. This trigger will display the salary difference between the old & new Salary.

CUSTOMERS(ID,NAME,AGE,ADDRESS,SALARY)

STEP Table 1: Create Database , Use DB , Create Table(salary_log)

```
CREATE TABLE salary_log (
    log_id INT AUTO_INCREMENT PRIMARY KEY,
    customer_id INT,
    old_salary DECIMAL(10,2),
    new_salary DECIMAL(10,2),
    salary_difference DECIMAL(10,2),
    action_type VARCHAR(10),
    log_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```
MySQL 8.0 Command Line Cli x + v
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
mysql> CREATE DATABASE trigger_db;
Query OK, 1 row affected (0.03 sec)

mysql> use trigger_db;
Database changed
mysql> CREATE TABLE salary_log (
    -> log_id INT AUTO_INCREMENT PRIMARY KEY,
    -> customer_id INT,
    -> old_salary DECIMAL(10,2),
    -> new_salary DECIMAL(10,2),
    -> salary_difference DECIMAL(10,2),
    -> action_type VARCHAR(10),
    -> log_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    -> );
Query OK, 0 rows affected (0.02 sec)

mysql> DESC SALARY_LOG;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| log_id | int | NO | PRI | NULL | auto_increment |
| customer_id | int | YES | | NULL | |
| old_salary | decimal(10,2) | YES | | NULL | |
| new_salary | decimal(10,2) | YES | | NULL | |
| salary_difference | decimal(10,2) | YES | | NULL | |
| action_type | varchar(10) | YES | | NULL | |
| log_time | timestamp | YES | | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
+-----+-----+-----+-----+-----+-----+
7 rows in set (0.00 sec)

mysql> |
```


STEP Table 2: Create CUSTOMERS Table

```
CREATE TABLE CUSTOMERS (
    ID INT PRIMARY KEY,
    NAME VARCHAR(50),
    AGE INT,
    ADDRESS VARCHAR(100),
    SALARY DECIMAL(10,2)
);
```

```
mysql> CREATE TABLE CUSTOMERS (
-> ID INT PRIMARY KEY,
-> NAME VARCHAR(50),
-> AGE INT,
-> ADDRESS VARCHAR(100),
-> SALARY DECIMAL(10,2)
-> );
Query OK, 0 rows affected (0.02 sec)

mysql> desc customers;
+-----+-----+-----+-----+-----+-----+
| Field | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| ID    | int           | NO   | PRI | NULL    |       |
| NAME  | varchar(50)   | YES  |     | NULL    |       |
| AGE   | int           | YES  |     | NULL    |       |
| ADDRESS | varchar(100) | YES  |     | NULL    |       |
| SALARY | decimal(10,2) | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
5 rows in set (0.00 sec)

mysql> |
```

INSERT Trigger

```
DELIMITER $$
```

```
CREATE TRIGGER customers_after_insert
AFTER INSERT ON CUSTOMERS
FOR EACH ROW
BEGIN
    INSERT INTO salary_log(customer_id, old_salary, new_salary,
salary_difference, action_type)
    VALUES(NEW.ID, NULL, NEW.SALARY, NEW.SALARY, 'INSERT');
END$$
```

```
DELIMITER ;
```

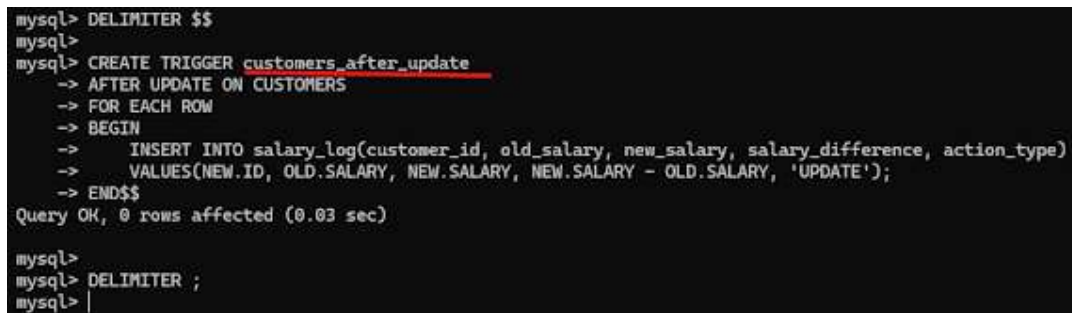
```
mysql> DELIMITER $$
mysql>
mysql> CREATE TRIGGER customers_after_insert
-> AFTER INSERT ON CUSTOMERS
-> FOR EACH ROW
-> BEGIN
-> INSERT INTO salary_log(customer_id, old_salary, new_salary, salary_difference, action_type)
-> VALUES(NEW.ID, NULL, NEW.SALARY, NEW.SALARY, 'INSERT');
-> END$$
Query OK, 0 rows affected (0.04 sec)
```

UPDATE Trigger

DELIMITER \$\$

```
CREATE TRIGGER customers_after_update
AFTER UPDATE ON CUSTOMERS
FOR EACH ROW
BEGIN
    INSERT INTO salary_log(customer_id, old_salary, new_salary,
salary_difference, action_type)
        VALUES (NEW.ID, OLD.SALARY, NEW.SALARY, NEW.SALARY -
OLD.SALARY, 'UPDATE');
END$$
```

DELIMITER ;



```
mysql> DELIMITER $$
mysql>
mysql> CREATE TRIGGER customers_after_update
-> AFTER UPDATE ON CUSTOMERS
-> FOR EACH ROW
-> BEGIN
->     INSERT INTO salary_log(customer_id, old_salary, new_salary, salary_difference, action_type)
->     VALUES(NEW.ID, OLD.SALARY, NEW.SALARY, NEW.SALARY - OLD.SALARY, 'UPDATE');
-> END$$
Query OK, 0 rows affected (0.03 sec)

mysql>
mysql> DELIMITER ;
mysql> |
```

DELETE Trigger

```
DELIMITER $$
```

```
CREATE TRIGGER customers_after_delete
```

```
AFTER DELETE ON CUSTOMERS
```

```
FOR EACH ROW
```

```
BEGIN
```

```
    INSERT INTO salary_log(customer_id, old_salary, new_salary,  
    salary_difference, action_type)
```

```
        VALUES(OLD.ID, OLD.SALARY, NULL, -OLD.SALARY, 'DELETE');
```

```
END$$
```

```
DELIMITER ;
```

```
mysql> DELIMITER ;
mysql> DELIMITER $$
mysql>
mysql> CREATE TRIGGER customers_after_delete
->
-> AFTER DELETE ON CUSTOMERS
->
-> FOR EACH ROW
->
-> BEGIN
->
->     INSERT INTO salary_log(customer_id, old_salary, new_salary, salary_difference, action_type)
->
->     VALUES(OLD.ID, OLD.SALARY, NULL, -OLD.SALARY, 'DELETE');
->
-> END$$
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> DELIMITER ;
mysql>
mysql> |
```

Test the Triggers**Insert Data:**

```
INSERT INTO CUSTOMERS VALUES (1, 'Jo', 30, 'New York', 40000);
```

```
mysql> INSERT INTO CUSTOMERS VALUES (1, 'Jo', 30, 'New York', 40000);
Query OK, 1 row affected (0.01 sec)
```

```
mysql> select *from customers;
```

```
+-----+-----+-----+-----+-----+
| ID | NAME | AGE | ADDRESS | SALARY |
+-----+-----+-----+-----+-----+
|  1 | Jo   |  30 | New York | 40000.00 |
+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

```
mysql> |
```

Update Salary:

```
UPDATE CUSTOMERS
SET SALARY = 50000
WHERE ID = 1;
```

```
mysql> select *from customers;
+-----+-----+-----+-----+-----+
| ID | NAME | AGE | ADDRESS | SALARY |
+-----+-----+-----+-----+-----+
| 1 | Jo | 30 | New York | 40000.00 |
+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

mysql> UPDATE CUSTOMERS
 -> SET SALARY = 50000
 -> WHERE ID = 1;
Query OK, 1 row affected (0.03 sec)
Rows matched: 1 Changed: 1 Warnings: 0

```
mysql> select *from customers;
+-----+-----+-----+-----+-----+
| ID | NAME | AGE | ADDRESS | SALARY |
+-----+-----+-----+-----+-----+
| 1 | Jo | 30 | New York | 50000.00 |
+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

mysql> |

Delete Record:

```
DELETE FROM CUSTOMERS WHERE ID = 1;
```

```
mysql> DELETE FROM CUSTOMERS WHERE ID = 1;
Query OK, 1 row affected (0.01 sec)
```

```
mysql> select *from customers;
Empty set (0.00 sec)
```

mysql> |

Check Salary Difference:

```
SELECT * FROM salary_log;
```

note:

You will see:

- Old salary
- New salary

- Salary difference
- Action type (INSERT / UPDATE / DELETE)

```
mysql> select *from customers;
Empty set (0.00 sec)

mysql> SELECT * FROM salary_log;
```

log_id	customer_id	old_salary	new_salary	salary_difference	action_type	log_time
1	1	NULL	40000.00	40000.00	INSERT	2026-02-11 15:06:18
2	1	40000.00	50000.00	10000.00	UPDATE	2026-02-11 15:09:16
3	1	50000.00	NULL	-50000.00	DELETE	2026-02-11 15:11:28

```
3 rows in set (0.00 sec)

mysql> |
```

EX: 5

Create cursor for Employee table & extract the values from the table. Declare the variables ,Open the cursor & extrect the values from the cursor. Close the cursor. Employee(E_id, E_name, Age, Salary)

1. Start the procedure

- Give the procedure a name.
- Begin the procedure block.

2. Declare variables

- Create variables to store column values from the table.
- All variables must be declared at the beginning.

3. Declare the cursor

- Define a cursor that selects rows from the table.
- This cursor will be used to read records one by one.

4. Declare a handler

- Create a handler to detect the end of the cursor.
- This handler will stop the loop when there are no more rows.

5. Open the cursor

- Open the cursor to start fetching rows.

6. Loop and fetch data

- Start a loop.
- Fetch one row from the cursor into variables.
- Check if the cursor reached the end.
- If not end, process the row (e.g., display or store values).

7. Exit the loop

- When no more rows exist, exit the loop.
-

8. Close the cursor

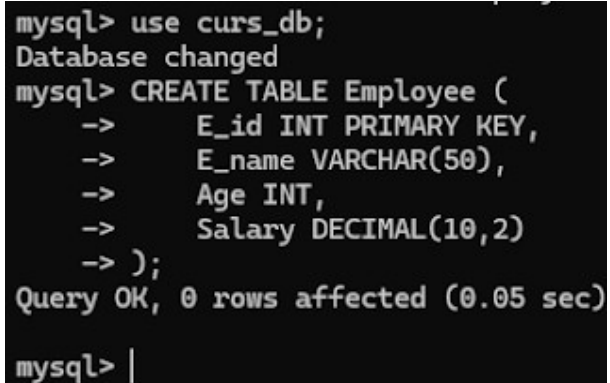
- Close the cursor to release resources.

9. End the procedure

- End the procedure block.
- Procedure is ready to be called.

CODE:**Step: Create DB, Use DB, Create Table.**

```
CREATE TABLE Employee (  
  
    E_id INT PRIMARY KEY,  
  
    E_name VARCHAR(50),  
  
    Age INT,  
  
    Salary DECIMAL(10,2)  
  
);
```



```
mysql> use curs_db;  
Database changed  
mysql> CREATE TABLE Employee (  
->     E_id INT PRIMARY KEY,  
->     E_name VARCHAR(50),  
->     Age INT,  
->     Salary DECIMAL(10,2)  
-> );  
Query OK, 0 rows affected (0.05 sec)  
  
mysql> |
```

Step D1: Insert rows:

```
mysql> INSERT INTO Employee VALUES (1, 'John', 30, 40000);
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO Employee VALUES (2, 'Alice', 28, 45000);
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO Employee VALUES (3, 'Bob', 35, 50000);
Query OK, 1 row affected (0.00 sec)

mysql> select *from employee;
+-----+-----+-----+-----+
| E_id | E_name | Age | Salary |
+-----+-----+-----+-----+
| 1    | John   | 30  | 40000.00 |
| 2    | Alice  | 28  | 45000.00 |
| 3    | Bob    | 35  | 50000.00 |
+-----+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> |
```

Create Stored Procedure with Cursor

Why stored procedure?

Because cursor logic requires a procedure.

Step E1: Change Delimiter

```
DELIMITER $$
```

Step E2: Create Procedure

```
CREATE PROCEDURE extract_employee()
BEGIN
```

Step E3: Declare Variables (All Declarations First)

```
DECLARE v_id INT;
DECLARE v_name VARCHAR(50);
DECLARE v_age INT;
DECLARE v_salary DECIMAL(10,2);
DECLARE done INT DEFAULT 0;
```

Step E4: Declare Cursor

```
DECLARE emp_cursor CURSOR FOR
    SELECT E_id, E_name, Age, Salary FROM Employee;
```

Step E5: Declare Handler

```
DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;
```

Step E6: Create Temporary Table

```
CREATE TEMPORARY TABLE IF NOT EXISTS temp_employee (
    E_id INT,
    E_name VARCHAR(50),
    Age INT,
    Salary DECIMAL(10,2)
);
```

Step E7: Open Cursor

```
OPEN emp_cursor;
```

Step E8: Fetch Loop

```
read_loop: LOOP
    FETCH emp_cursor INTO v_id, v_name, v_age, v_salary;
```



```

        IF done = 1 THEN
            LEAVE read_loop;
        END IF;

        INSERT INTO temp_employee VALUES (v_id, v_name, v_age, v_salary);
    END LOOP;

```

Step E9: Close Cursor

```
CLOSE emp_cursor;
```

Step E10: Display Data

```
SELECT * FROM temp_employee;
```

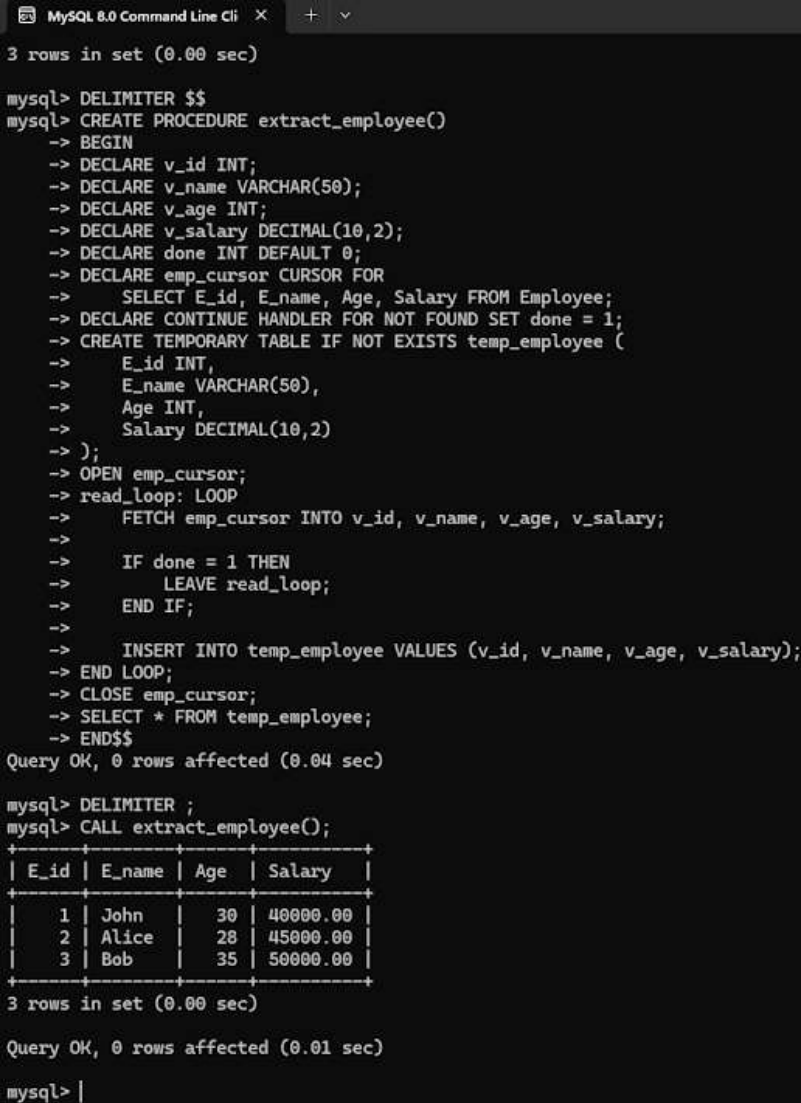
Step E11: End Procedure

```
END$$
```

```
DELIMITER ;
```

Run the Procedure

```
CALL extract_employee();
```



```

MySQL 8.0 Command Line Cli x + v
3 rows in set (0.00 sec)

mysql> DELIMITER $$
mysql> CREATE PROCEDURE extract_employee()
-> BEGIN
-> DECLARE v_id INT;
-> DECLARE v_name VARCHAR(50);
-> DECLARE v_age INT;
-> DECLARE v_salary DECIMAL(10,2);
-> DECLARE done INT DEFAULT 0;
-> DECLARE emp_cursor CURSOR FOR
-> SELECT E_id, E_name, Age, Salary FROM Employee;
-> DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;
-> CREATE TEMPORARY TABLE IF NOT EXISTS temp_employee (
-> E_id INT,
-> E_name VARCHAR(50),
-> Age INT,
-> Salary DECIMAL(10,2)
-> );
-> OPEN emp_cursor;
-> read_loop: LOOP
-> FETCH emp_cursor INTO v_id, v_name, v_age, v_salary;
->
-> IF done = 1 THEN
-> LEAVE read_loop;
-> END IF;
->
-> INSERT INTO temp_employee VALUES (v_id, v_name, v_age, v_salary);
-> END LOOP;
-> CLOSE emp_cursor;
-> SELECT * FROM temp_employee;
-> END$$
Query OK, 0 rows affected (0.04 sec)

mysql> DELIMITER ;
mysql> CALL extract_employee();
+-----+-----+-----+-----+
| E_id | E_name | Age | Salary |
+-----+-----+-----+-----+
| 1 | John | 30 | 40000.00 |
| 2 | Alice | 28 | 45000.00 |
| 3 | Bob | 35 | 50000.00 |
+-----+-----+-----+-----+
3 rows in set (0.00 sec)

Query OK, 0 rows affected (0.01 sec)

mysql> |

```

EX NO:6

Write a PL/SQL block of code using parameterized Cursor, that will merge the data available in the newly created table N_RollCall with the data available in the table O_RollCall. If the data in the first table already exist in the second table then that data should be skipped.

Meaning of PL/SQL Block

A **PL/SQL Block** is a structured group of statements written in **PL/SQL (Procedural Language/Structured Query Language)** that are executed together as a single unit.

It allows you to combine:

- SQL statements (like SELECT, INSERT, UPDATE, DELETE)
- Procedural statements (like loops, conditions, variables, exception handling)

Meaning of the Given PL/SQL Program

This PL/SQL block is written in **Oracle Database**.

Purpose of the Program

The program **merges data from table N_RollCall (New Roll Call) into O_RollCall (Old Roll Call)**.

- 1, If a record from N_RollCall already exists in O_RollCall, it is **skipped**.
- 2, If it does not exist, it is **inserted** into O_RollCall.

STEP1: CREATE TABLE **N_RollCall:**

```
CREATE TABLE N_RollCall (  
    id INT,  
    name VARCHAR(100),  
    roll_date DATE  
);
```

```
mysql> use dan_db;
Database changed
mysql> CREATE TABLE N_RollCall (
    ->     id INT,
    ->     name VARCHAR(100),
    ->     roll_date DATE
    -> );
Query OK, 0 rows affected (0.04 sec)

mysql> |
```

STEP2: INSERT DATA in N_RollCall Table:

```
INSERT INTO N_RollCall VALUES (1, 'RAJ', '2015-01-22');
INSERT INTO N_RollCall VALUES (2, 'KING', '2016-04-22');
INSERT INTO N_RollCall VALUES (3, 'RUTH', '2017-05-22');
INSERT INTO N_RollCall VALUES (4, 'BABU', '2017-05-22');
```

```
mysql> INSERT INTO N_RollCall VALUES (1, 'RAJ', '2015-01-22');
Query OK, 1 row affected (0.04 sec)

mysql> INSERT INTO N_RollCall VALUES (2, 'KING', '2016-04-22');
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO N_RollCall VALUES (3, 'RUTH', '2017-05-22');
Query OK, 1 row affected (0.00 sec)

mysql> |
```

STEP3: CREATE TABLE O_RollCall:

```
CREATE TABLE O_RollCall (

    id INT,

    name VARCHAR(100),

    roll_date DATE

);
```

STEP4: INSERT DATA in O_RollCall Table:

```
INSERT INTO O_RollCall VALUES (1, 'BABU', '2015-01-22');
INSERT INTO O_RollCall VALUES (2, 'KRISH', '2016-04-22');
INSERT INTO O_RollCall VALUES (3, 'JONA', '2017-05-22');
```

```
mysql> CREATE TABLE O_RollCall (
->     id INT,
->     name VARCHAR(100),
->     roll_date DATE
-> );
Query OK, 0 rows affected (0.05 sec)

mysql> INSERT INTO O_RollCall VALUES (1, 'BABU', '2015-01-22');
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO O_RollCall VALUES (2, 'KRISH', '2016-04-22');
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO O_RollCall VALUES (3, 'JONA', '2017-05-22');
Query OK, 1 row affected (0.01 sec)

mysql> |
```

Step 5: Merge N_RollCall into O_RollCall while skipping duplicate names

```
INSERT INTO O_RollCall (id, name, roll_date)

SELECT n.id, n.name, n.roll_date

FROM N_RollCall n

WHERE NOT EXISTS (

    SELECT 1

    FROM O_RollCall o

    WHERE o.name = n.name

);
```

```
mysql> SELECT * FROM O_RollCall;
+----+-----+-----+
| id | name  | roll_date |
+----+-----+-----+
| 1  | BABU  | 2015-01-22 |
| 2  | KRISH | 2016-04-22 |
| 3  | JONA  | 2017-05-22 |
| 1  | RAJ   | 2015-01-22 |
| 2  | KING  | 2016-04-22 |
| 3  | RUTH  | 2017-05-22 |
| 4  | BABU  | 2017-05-22 |
+----+-----+-----+
7 rows in set (0.00 sec)
```

Step 6: Optional – Remove Existing Duplicate Names First

```
DELETE o1
FROM O_RollCall o1
JOIN O_RollCall o2
ON o1.name = o2.name
AND o1.id > o2.id;
```

Step 7: Optional – Add UNIQUE Constraint to Prevent Future Duplicates

```
ALTER TABLE O_RollCall
ADD CONSTRAINT unique_name UNIQUE (name);
```

Step 8: View the Final Table

```
SELECT * FROM O_RollCall;
```

```
MySQL 8.0 Command Line Cli x + v

mysql> SELECT * FROM O_RollCall;
+----+-----+-----+
| id | name | roll_date |
+----+-----+-----+
| 1  | BABU | 2015-01-22 |
| 2  | KRISH | 2016-04-22 |
| 3  | JONA | 2017-05-22 |
| 1  | RAJ  | 2015-01-22 |
| 2  | KING | 2016-04-22 |
| 3  | RUTH | 2017-05-22 |
| 4  | BABU | 2017-05-22 |
+----+-----+-----+
7 rows in set (0.00 sec)

mysql> INSERT INTO O_RollCall (id, name, roll_date)
-> SELECT n.id, n.name, n.roll_date
-> FROM N_RollCall n
-> WHERE NOT EXISTS (
->   SELECT 1
->   FROM O_RollCall o
->   WHERE o.name = n.name
-> );
Query OK, 0 rows affected (0.00 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> DELETE o1
-> FROM O_RollCall o1
-> JOIN O_RollCall o2
-> ON o1.name = o2.name
-> AND o1.id > o2.id;
Query OK, 1 row affected (0.01 sec)

mysql> ALTER TABLE O_RollCall
-> ADD CONSTRAINT unique_name UNIQUE (name);
Query OK, 0 rows affected (0.03 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM O_RollCall;
+----+-----+-----+
| id | name | roll_date |
+----+-----+-----+
| 1  | BABU | 2015-01-22 |
| 2  | KRISH | 2016-04-22 |
| 3  | JONA | 2017-05-22 |
| 1  | RAJ  | 2015-01-22 |
| 2  | KING | 2016-04-22 |
| 3  | RUTH | 2017-05-22 |
+----+-----+-----+
6 rows in set (0.00 sec)

mysql> |
```

X - Not Repeated

Ex1 viva

1. What is MySQL?
MySQL is a **Relational Database Management System (RDBMS)** used to store and manage data.
 2. What is a table?
A table is a collection of **rows and columns** used to store data in a database.
 3. What is a primary key?
A primary key is a column that **uniquely identifies each record** in a table.
 4. Can a primary key contain NULL?
No, a primary key **cannot contain NULL values**.
 5. What is NOT NULL constraint?
It ensures that a column **cannot store NULL values**.
 6. What is DDL?
DDL (Data Definition Language) is used to **define database structure**.
Example: CREATE, ALTER, DROP.
 7. What is DML?
DML (Data Manipulation Language) is used to **manipulate data**.
Example: INSERT, UPDATE, DELETE.
 8. What is TCL?
TCL (Transaction Control Language) is used to **manage transactions**.
Example: COMMIT, ROLLBACK.
 9. What is ROLLBACK?
ROLLBACK is used to **undo changes** made in a transaction.
 10. What is COMMIT?
COMMIT is used to **save changes permanently** in the database.
 11. What is a transaction?
A transaction is a **group of SQL statements executed as one unit**.
 12. What is VARCHAR?
VARCHAR is a **variable-length string data type**.
 13. Difference between DELETE and DROP?
DELETE removes records from a table.

DROP deletes the entire table structure.
 14. What is GRANT command?
GRANT is used to **give permissions** to a user.
-

15. What is FLUSH PRIVILEGES?

It reloads the **user permission tables** in MySQL.

16. What is ALTER TABLE?

ALTER TABLE is used to **modify an existing table structure**.

17. What is SELECT statement?

SELECT is used to **retrieve data** from a table.

18. What is a constraint?

A constraint is a rule that **restricts the type of data** entered into a table.

19. What is AUTO_INCREMENT?

AUTO_INCREMENT automatically **generates unique numbers** for a column.

20. What happens if primary key is duplicated?

The database will show an **error** because primary key must be unique.

Ex no 2 : viva question

1. What is a database?

A database is an organized collection of data that is stored and managed electronically. It allows users to easily access, manage, and update information.

2. What is a table in DBMS?

A table is a structure in a database that stores data in the form of rows and columns. Each row represents a record and each column represents a field.

3. What is the purpose of CREATE TABLE command?

The CREATE TABLE command is used to create a new table in a database. It defines the table name, column names, and data types of each column.

4. What is ALTER TABLE used for?

The ALTER TABLE command is used to modify the structure of an existing table. It can add, delete, or modify columns in a table.

5. What is the use of ADD keyword in SQL?

The ADD keyword is used with ALTER TABLE to add a new column or constraint to an existing table.

6. What is a domain in database?

A domain is the set of valid values that a column can contain. It defines the data type and possible range of values for a column.

7. What is INSERT command?

The INSERT command is used to add new records into a table in a database.

8. What is UPDATE command?

The UPDATE command is used to modify existing records in a table based on a condition.

9. What is DELETE command?

The DELETE command is used to remove one or more records from a table based on a specified condition.

10. What is the difference between DELETE and DROP?

DELETE removes specific records from a table, while DROP removes the entire table structure from the database.

11. What is VARCHAR data type?
VARCHAR is a variable-length character data type used to store text or string values in a table.
12. What is DECIMAL data type?
DECIMAL is a numeric data type used to store exact numeric values with fixed precision and scale, such as salary or commission.
13. What is a column in a table?
A column is a vertical structure in a table that represents a specific attribute or field of the data.
14. What is a row in a table?
A row is a horizontal structure in a table that represents a single record containing related data.
15. What is the purpose of WHERE clause?
The WHERE clause is used to specify conditions when retrieving, updating, or deleting records from a table.
16. What happens if WHERE clause is not used in UPDATE?
If the WHERE clause is not used in UPDATE, all records in the table will be updated.
17. What happens if WHERE clause is not used in DELETE?
If the WHERE clause is not used in DELETE, all records in the table will be deleted.
18. What is DDL?
DDL stands for Data Definition Language. It is used to define and manage the structure of database objects such as tables. Examples include CREATE, ALTER, and DROP.
19. What is DML?
DML stands for Data Manipulation Language. It is used to manipulate data stored in database tables. Examples include INSERT, UPDATE, and DELETE.
20. Why is SQL important?
SQL is important because it is the standard language used to communicate with databases. It helps in storing, retrieving, and managing data efficiently.

Ex no 3 : viva

1. What is an aggregate function?
An aggregate function is a function that performs a calculation on multiple rows and returns a single value.
2. What is COUNT function?
COUNT function is used to count the number of rows or values in a column.
3. What is AVG function?
AVG function is used to calculate the average value of a numeric column.
4. What is MAX function?
MAX function returns the highest value from a column.
5. What is MIN function?
MIN function returns the lowest value from a column.
6. What is SUM function?
SUM function is used to calculate the total of a numeric column.
7. What is GROUP BY clause?
GROUP BY clause is used to group rows that have the same values in specified columns.
8. Why is GROUP BY used?
GROUP BY is used to apply aggregate functions to groups of rows.

9. What is ORDER BY clause?
ORDER BY clause is used to sort the result in ascending or descending order.
10. What is the default order of ORDER BY?
The default order is ascending (ASC).
11. What is the difference between WHERE and GROUP BY?
WHERE filters rows before grouping, while GROUP BY groups rows after filtering.
12. Can aggregate functions be used without GROUP BY?
Yes, aggregate functions can be used without GROUP BY when calculating a single result for the entire table.
13. What type of data is used with SUM and AVG?
SUM and AVG are used with numeric data types.
14. What happens if GROUP BY is not used with multiple columns in SELECT?
An error may occur if non-aggregated columns are selected without GROUP BY.
15. Why is SQL important in data analysis?
SQL is important because it helps in retrieving, summarizing, and analyzing data efficiently.

Ex no 4 : viva

1. What is a trigger?
A trigger is a stored program in the database that automatically executes when a specific event occurs on a table.
2. What is a row level trigger?
A row level trigger is a trigger that executes once for each row affected by an INSERT, UPDATE, or DELETE operation.
3. When does a trigger execute?
A trigger executes automatically before or after an INSERT, UPDATE, or DELETE operation.
4. What are the types of triggers?
The main types are BEFORE trigger and AFTER trigger.
5. What is the use of FOR EACH ROW?
FOR EACH ROW specifies that the trigger will execute for every affected row.
6. What is OLD keyword in trigger?
OLD refers to the previous value of a column before modification or deletion.
7. What is NEW keyword in trigger?
NEW refers to the new value of a column after insertion or update.
8. Can we use trigger for INSERT operation?
Yes, triggers can be created for INSERT operation.
9. Can we use OLD in INSERT trigger?
No, OLD is not available in INSERT because there is no previous record.
10. Can we use NEW in DELETE trigger?
No, NEW is not available in DELETE because the record is removed.
11. What is the purpose of trigger?
The purpose of a trigger is to maintain data integrity and automate database operations.
12. What is the difference between procedure and trigger?
A procedure is executed manually, while a trigger is executed automatically.
13. What is BEFORE trigger?
A BEFORE trigger executes before the actual database operation.

14. What is AFTER trigger?

An AFTER trigger executes after the database operation is completed.

15. Why are triggers important in DBMS?

Triggers are important because they enforce rules, maintain consistency, and automate tasks in the database.

Ex no 5 : viva

1. What is a cursor in DBMS?

A cursor is a database object used to retrieve and process rows one by one from a result set.

2. Why do we use a cursor?

A cursor is used when we need to process records individually instead of all at once.

3. Where is a cursor used in MySQL?

A cursor is used inside stored procedures.

4. What are the steps to use a cursor?

The steps are Declare cursor, Open cursor, Fetch data, and Close cursor.

5. What is DECLARE statement in cursor?

DECLARE is used to define variables and the cursor.

6. What does OPEN cursor do?

OPEN cursor starts the execution of the cursor and prepares it to fetch rows.

7. What is FETCH in cursor?

FETCH retrieves the current row from the cursor into variables.

8. Why do we close a cursor?

We close a cursor to release the resources used by it.

9. What is a handler in cursor?

A handler is used to manage conditions such as when no more rows are found.

10. What is NOT FOUND in cursor?

NOT FOUND indicates that there are no more rows to fetch.

11. Can we use cursor outside a stored procedure?

No, in MySQL cursors are used only inside stored procedures.

12. What is the difference between cursor and SELECT statement?

SELECT retrieves all records at once, while cursor processes records one by one.

13. Is cursor faster than SELECT?

No, cursors are slower because they process rows individually.

14. What is LOOP in cursor?

LOOP is used to repeatedly fetch rows until all rows are processed.

15. Why are cursors important?

Cursors are important when row-by-row processing is required in a database program.

Ex no 6 : viva

1. What is PL/SQL?

PL/SQL is a procedural language extension of SQL used in Oracle database to write programs.

2. What is a cursor in PL/SQL?

A cursor is a pointer that retrieves rows from a query result one by one.

3. What is a parameterized cursor?

A parameterized cursor is a cursor that accepts parameters to retrieve specific records.

4. Why do we use parameterized cursor?
It is used to pass values dynamically to a query inside the cursor.
5. What is the difference between simple cursor and parameterized cursor?
A simple cursor does not accept parameters, while a parameterized cursor accepts parameters.
6. What is %TYPE in PL/SQL?
%TYPE is used to declare a variable with the same data type as a table column.
7. What is SELECT INTO statement?
SELECT INTO is used to retrieve data from a table and store it into variables.
8. What is the purpose of COUNT() *in this program*?
COUNT() is used to check whether a record already exists in the table.
9. What is COMMIT in PL/SQL?
COMMIT saves the changes permanently in the database.
10. What is the purpose of IF condition in PL/SQL?
IF condition is used to execute statements based on a condition.
11. What is a loop in PL/SQL?
A loop is used to execute a block of statements repeatedly.
12. What is the use of OPEN cursor?
OPEN cursor starts the execution of the cursor query.
13. What is FETCH in cursor?
FETCH retrieves data from the cursor into variables.
14. Why do we CLOSE a cursor?
We close a cursor to release memory and resources.
15. Why is PL/SQL important?
PL/SQL is important because it allows combining SQL with programming features like loops and conditions.

Ex no 7 : viva

1. What is MongoDB?
MongoDB is an open-source NoSQL database that stores data in document format using JSON-like structure.
 2. What type of database is MongoDB?
MongoDB is a NoSQL, document-oriented database.
 3. What is a document in MongoDB?
A document is a set of key-value pairs stored in BSON format.
 4. What is a collection in MongoDB?
A collection is a group of MongoDB documents, similar to a table in SQL.
 5. What is CRUD in MongoDB?
CRUD stands for Create, Read, Update, and Delete operations.
 6. Which command is used to insert data in MongoDB?
insertOne() and insertMany() are used to insert data.
 7. How do you retrieve data in MongoDB?
The find() method is used to retrieve data.
 8. How do you update data in MongoDB?
updateOne() or updateMany() methods are used to update data.
 9. How do you delete data in MongoDB?
deleteOne() or deleteMany() methods are used to delete data.
 10. What is the difference between SQL and MongoDB?
SQL uses tables and rows, while MongoDB uses collections and documents.
-

11. What is BSON?

BSON is Binary JSON format used to store documents in MongoDB.

12. What is the default port number of MongoDB?

The default port number is 27017.

13. What is use command in MongoDB?

The use command is used to create or switch a database.

14. Is MongoDB schema-less?

Yes, MongoDB is schema-less, meaning structure can vary between documents.

15. Why is MongoDB popular?

MongoDB is popular because it is flexible, scalable, and handles large amounts of unstructured data efficiently.